



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МИИ»

Инс	ИВТИ
титут	
Каф	МКМ
едра	

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ
РАБОТА БАКАЛАВРА**

Направл ение	01.03.02 Прикладная математика и информатика (код и наименование)			
Образовательная программа	Математическое моделирование			
Форма обучения	очная (очная/очно-заочная/заочная)			
Т ема:	Проектирование и реализация системы централизованного логирования распределённых приложений			
Ст удент	А-16-22 группа	подпись	Жунусов Е.Е. фамилия и инициалы	
Руководит ель ВКР	уч. степень	старший преподаватель должность	п одпись	Шевченко И.В. фамилия и инициалы
Консульта нт	уч. степень	должность	п одпись	фамилия и инициалы
Внешний консультант	уч. степень	должность	п одпись	фамилия и инициалы
организация				
«Работа допущена к защите»				
Заведую щий кафедрой	к.ф.- м.н.	доце нт	Зубков П.В. фамилия и инициалы	
	уч. степень	зван ие	подпис ь	

Дата

Москва, 2026



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Инс	ИВТИ
титут	
Каф	МКМ
едра	

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА

Направление	01.03.02 Прикладная математика и информатика			
	(код и наименование)			
Образовательная программа	Математическое моделирование			
Форма обучения	очная			
	(очная/очно-заочная/заочная)			
Тема:	Проектирование и реализация системы централизованного логирования распределённых приложений			
Студент	А-16-22	Жунусов Е.Е.		
	группа	подпись	фамилия и инициалы	
Руководитель ВКР		старший преподаватель	Шевченко И.В.	
	уч. степень	должность	подпись	фамилия и инициалы
Консультант				
	уч. степень	должность	подпись	фамилия и инициалы
Внешний консультант				
	уч. степень	должность	подпись	фамилия и инициалы
организация				

План выполнения выпускной квалификационной работы

№ п/п	Содержание разделов	Срок в ыпол- н ения	Тру- доём- кос- ть, %
I			
I.			
II.			
V.	Оформление ВКР		

Консультации по ВКР (консультант)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дат
		а

Консультации по ВКР (консультант 2)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дат
		а

Консультации по ВКР (внешний консультант)

Перечень разделов и заданий:

_____	_____	_____
фамилия, имя, отчество	подпись	дат
		а

Дата выдачи задания:	_____
	09.02.2026
	дата

Студент:

Жунусов Евгений Ерланович

_____	_____	_____
фамилия, имя, отчество	подпись	дат
		а

Руководитель ВКР:

Шевченко Иван Владимирович

_____	_____	_____
фамилия, имя, отчество	подпись	дат
		а

Примечания:

1. Задание брошюруется вместе с выпускной работой после титульного листа (страницы задания имеют номера 2, 3, 4, 5).
2. Отзыв руководителя, рецензия(и), отчет о проверке на объем заимствований и согласие студента на размещение работы в открытом доступе вкладываются в конверт (файловую папку) под обложкой работы.

АННОТАЦИЯ

Жунусов Е.Е. Проектирование и реализация системы централизованного логирования распределённых приложений. Выпускная квалификационная работа бакалавра. — Москва: НИУ «МЭИ», 2026. — 50 с., 4 рис., 8 табл., 4 прил.

В работе спроектирована и реализована система централизованного логирования распределённых приложений для тренажёрной системы. Сервер принимает сигналы и события через WebSocket, пакетно сохраняет их в ClickHouse, предоставляет REST API для чтения, архивирует завершённые сессии в JSON Lines с gzip-сжатием и удаляет данные по запросу. Часовое испытание подтвердило обработку 17 995 850 сигналов и 719 665 событий без потерь.

Ключевые слова: централизованное логирование, распределённые приложения, тренажёрная система, WebSocket, REST API, ClickHouse, Go, буферизация, архивирование, нагрузочное тестирование.

Annotation

Zhunusov E.E. Design and implementation of a centralized logging system for distributed applications. Bachelor's final qualification work. — Moscow: NRU MPEI, 2026. — 50 pages, 4 figures, 8 tables, 4 appendices.

The thesis presents a centralized logging system for distributed applications intended for a training simulator. The server receives signals and events through WebSocket, writes them to ClickHouse in batches, provides REST API access, creates gzip-compressed JSON Lines archives, and deletes data on request. A one-hour test confirmed 17,995,850 signals and 719,665 events without loss.

Keywords: centralized logging, distributed applications, training simulator, WebSocket, REST API, ClickHouse, Go, buffering, archiving, load testing.

ВВЕДЕНИЕ

Современные информационные системы всё чаще состоят не из одной программы, а из набора взаимодействующих сервисов, клиентских модулей и хранилищ. В такой среде результат работы зависит не только от бизнес-логики, но и от того, насколько подробно зафиксированы действия отдельных компонентов. Поэтому данные о ходе выполнения приложения имеют не меньшее значение, чем сами бизнес-данные. Если раньше журналирование часто воспринималось как вспомогательный механизм для вывода сообщений об ошибках, то в распределённых приложениях оно становится одним из основных способов понимания поведения системы. Логи позволяют восстановить последовательность событий, определить причину сбоя, оценить действия пользователя, выявить перегрузку отдельных компонентов и получить материал для последующего анализа [1, 2].

Актуальность темы связана с тем, что распределённые приложения редко существуют как единый исполняемый процесс. Они состоят из нескольких подсистем, сетевых взаимодействий, клиентских компонентов, серверных сервисов и хранилищ данных. Каждый компонент может формировать собственные сообщения. Если эти сообщения остаются только в локальных файлах или временной памяти отдельных узлов, общая картина работы приложения быстро распадается. При возникновении проблемы специалист вынужден вручную собирать данные с разных машин, сопоставлять временные отметки, искать нужный фрагмент и проверять, не были ли логи удалены или перезаписаны.

Особенно остро эта проблема проявляется в тренажёрных системах. Тренажёрная система не просто обслуживает отдельные запросы пользователя, а моделирует длительный процесс обучения или проверки действий оператора. В ходе одной тренировки могут формироваться значения параметров модели, события интерфейса, действия пользователя, системные уведомления, аварийные сообщения, сведения о перезапусках и технические данные о состоянии приложения. Такие данные важны не только для отладки, но и для анализа качества

выполнения тренировки. По ним можно понять, когда оператор совершил действие, как изменилась модель, какие события возникли до и после этого действия, в какой момент произошёл переход системы в новое состояние.

При этом объём таких данных может быть значительным. Если тренажёр с высокой частотой передаёт значения параметров модели, то за одну тренировочную сессию может накопиться несколько сотен мегабайт информации. Данные нельзя бесконечно хранить в активной базе без учёта их жизненного цикла. С одной стороны, они должны быть доступны после завершения тренировки, потому что именно тогда часто проводится разбор результатов. С другой стороны, после завершения анализа хранение в активном виде становится избыточным и приводит к росту объёма хранилища. Поэтому задача централизованного логирования для тренажёрных систем включает не только приём и запись данных, но и архивирование, последующее чтение из архива и полное удаление устаревших сессий.

Объектом исследования является процесс централизованного сбора, хранения и обработки журналов событий распределённых приложений. Предметом исследования являются методы и программные средства проектирования системы, которая принимает поток логов от внешнего источника, сохраняет их в структурированном виде, предоставляет программный интерфейс доступа к ним и управляет их жизненным циклом.

Целью работы является проектирование и реализация системы централизованного логирования распределённых приложений, предназначенной для приёма логов тренажёрной системы, их хранения, получения для анализа, архивирования с целью экономии дискового пространства и удаления по запросу.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать особенности логирования в распределённых приложениях и тренажёрных системах.
2. Определить состав данных, поступающих от тренажёра в процессе выполнения тренировочной сессии.

3. Рассмотреть существующие подходы к централизованному логированию и определить их применимость к поставленной задаче.
4. Обосновать выбор клиент-серверной архитектуры, транспортного протокола и хранилища данных.
5. Спроектировать WebSocket API для потокового приёма сообщений и REST API для управления данными.
6. Спроектировать модель данных, схему хранения и механизм буферизации записей.
7. Реализовать серверную часть системы на языке Go.
8. Реализовать хранение логов в ClickHouse, архивирование в сжатый формат и удаление данных.
9. Разработать инструмент нагрузочного тестирования, имитирующий работу нескольких тренажёрных сессий.
10. Оценить работоспособность решения и определить направления дальнейшего развития.

Практическая значимость работы заключается в создании самостоятельного программного компонента, который может быть интегрирован с тренажёрным комплексом и использоваться как централизованное хранилище логов. Такая система позволяет отделить основную логику тренажёра от задач долговременного хранения данных, обеспечивает единый интерфейс доступа к событиям и даёт возможность уменьшать объём активного хранилища за счёт архивирования.

1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ

1.1. Источник данных

Источником логов является тренажёрное приложение. Во время одной тренировки пользователь может несколько раз перезапустить модель, поэтому одной временной отметки недостаточно. Каждая запись содержит `session_id`, который связывает её с тренировкой, и `restart_id`, который отделяет один запуск модели от другого.

Для сигналов используется модельное время `model_time`. Оно не совпадает с системным временем компьютера: модель можно остановить, ускорить или перезапустить. При анализе тренировки записи сортируются сначала по `restart_id`, затем по `model_time`.

Событийные данные поступают реже сигналов, но содержат контекст действий оператора. В поле `data` хранится JSON-строка с дополнительными атрибутами. Например, событие может содержать область тренажёра, тег элемента, текст сообщения и текущее значение параметра.

Объём одной сессии зависит от частоты сигналов и длительности тренировки. Если один тренажёр передаёт 500 сигналов в секунду в течение часа, сервер получит 1,8 млн сигнальных записей. При десяти одновременно работающих тренажёрах это уже 18 млн записей за час. Поэтому схема должна быть рассчитана не на единичные сообщения, а на длительный последовательный поток.

Для анализа важна связь двух типов данных. Событие само по себе показывает действие, но не показывает реакцию модели. Сигналы показывают изменение параметров, но не объясняют причину. Сопоставление по `session_id`, `restart_id` и `model_time` позволяет восстановить последовательность: действие оператора, возникшее событие и изменение значений модели.

1.2. Типы хранимых данных

В системе используются три сущности:

1. Session — метаданные тренировки: идентификатор, время начала, время окончания и статус.
2. SignalLog — числовое значение параметра модели.
3. EventLog — действие пользователя или системное событие.

Разделение сигналов и событий сделано намеренно. Сигнал всегда содержит числовое значение и имеет стабильный набор полей. Событие может иметь разный состав дополнительных данных. Если хранить оба типа в одной таблице, большая часть столбцов будет пустовать, а запросы станут менее понятными.

1.3. Функциональные требования

Сервер должен выполнять полный цикл работы с данными: принять сообщение, сохранить его, вернуть по запросу, перенести в архив и удалить. Соответствие требований компонентам проекта показано в таблице 1.1.

Таблица 1.1 - Соответствие требований и компонентов реализации

Требование	Компонент	Результат
Потоковый приём	handler.wsStream	WebSocket-соединение на одну сессию
Разделение данных	SignalLog и EventLog	Отдельные модели, буферы и таблицы
Пакетная запись	service.flushSignals / flushEvents	Порог 500 записей, таймер 1 с
Чтение	GET signal-logs / event-logs	limit, offset, restart_id для активных данных
Архивирование	ArchiveSession	Два файла JSONL.GZ на сессию
Удаление	DeleteSession	Партиции, метаданные, файлы и буферы

Важное правило целостности: идентификатор сессии берётся из URL WebSocket-соединения. Поле session_id внутри сообщения перезаписывается сервером. Благодаря этому клиент не может случайно отправить запись в другую сессию.

1.4. Нефункциональные требования

К системе предъявлены следующие требования:

1. Одновременная работа нескольких WebSocket-соединений.
2. Пакетная запись вместо отдельного SQL-запроса на каждое сообщение.
3. Возможность запуска на другой машине без ручной установки компонентов.
4. Отделение сетевой логики, бизнес-логики и доступа к базе данных.
5. Сохранение доступа к данным после архивирования.
6. Проверка API-ключа для операций чтения и управления.
7. Понятная диагностика состояния системы через серверный журнал.

Реализация ориентирована на контролируемые сценарии тестирования. Буферизация в оперативной памяти выбрана как простой и быстрый механизм для демонстрационной реализации, а её устойчивость проверяется экспериментально.

Под производительностью в этой работе понимается не максимальное число сообщений, которое теоретически может принять Go-сервер. Проверяется более практический показатель: сколько сообщений генератор смог отправить за заданное время и сколько из них затем найдено в ClickHouse. Такой критерий одновременно учитывает сеть, обработчик, блокировки, буферизацию и запись в базу.

1.5. Критерии проверки

Работоспособность считается подтверждённой, если:

1. Сервер запускается вместе с ClickHouse через Docker Compose.
2. Маршрут /health возвращает код 200.
3. WebSocket-клиент передаёт сигналы и события нескольких сессий.
4. Количество записей в ClickHouse совпадает с количеством успешно отправленных сообщений.
5. REST API возвращает данные с фильтрацией и страничными параметрами.
6. После архивирования создаются два gzip-файла, а чтение через REST API продолжает работать.

7. После удаления исчезают таблицы данных конкретной сессии, метаданные и архивные файлы.

1.6. Выводы по первой главе

Основной единицей системы является тренировочная сессия. Все операции выполняются по `session_id`: подключение, запись, чтение, архивирование и удаление. Поток разделён на сигналы и события, а порядок внутри тренировки задают `restart_id` и `model_time`. Эти решения напрямую отражены в моделях Go и таблицах ClickHouse.

2. ВЫБОР ТЕХНИЧЕСКОГО РЕШЕНИЯ

2.1. Готовая платформа или отдельный сервис

Elasticsearch, Loki и Graylog решают общую задачу централизованного сбора журналов. Однако проекту требуется не только поиск текста. Необходимо принимать числовые сигналы, группировать их по тренировке, учитывать перезапуски модели и архивировать данные всей сессии одной операцией.

Использование готовой платформы добавило бы несколько компонентов и усложнило демонстрацию. Поэтому выбран отдельный сервис с небольшим API. Он не заменяет промышленную платформу наблюдаемости, но реализует нужный для тренажёра цикл данных без лишних функций.

Отказ от готовой платформы также позволяет показать собственную реализацию ключевых механизмов: разбор сообщений, управление сессиями, пакетную вставку, перенос данных между активным и архивным слоями. Для дипломной работы это важнее, чем настройка большого набора готовых компонентов.

2.2. Выбор транспорта

Рассматривались обычные HTTP-запросы, gRPC и WebSocket. Сравнение приведено в таблице 2.1.

Таблица 2.1 - Сравнение способов передачи логов

Вариант	Преимущества	Недостатки	Решение
HTTP POST	Простая реализация	Запрос на каждую запись или клиентский буфер	Не выбран
gRPC stream	Строгая схема, эффективный поток	Генерация кода и дополнительная настройка	Не выбран
WebSocket	Постоянное соединение, JSON, поддержка ping/pong	Нужно управлять долгим соединением	Выбран

WebSocket выбран потому, что тренировка естественно представляется долгоживущим соединением. Клиент открывает соединение при начале сессии, передаёт сообщения и завершает его командой `session_end`. Новый HTTP-запрос на каждую запись не создаётся. Формат сообщений остаётся простым JSON, что облегчает интеграцию с тренажёром [3].

REST используется отдельно для операций, которым не требуется постоянное соединение: чтения логов, архивирования и удаления.

2.3. Выбор хранилища

PostgreSQL подходит для метаданных и транзакционных операций, но поток сигналов в этой работе имеет аналитический характер: данные в основном добавляются и читаются большими диапазонами. ClickHouse рассчитан на такие вставки и выборки [4].

В ClickHouse записи одной таблицы физически организованы частями. Частые одиночные вставки создают много небольших частей и увеличивают нагрузку на фоновые слияния. По этой причине выбор ClickHouse связан с обязательной буферизацией: база эффективна только тогда, когда приложение передаёт ей блоки данных, а не каждое значение отдельно.

В проекте полезны следующие возможности ClickHouse:

1. Движок MergeTree для таблиц логов.
2. Колоночное хранение повторяющихся полей.
3. LowCardinality(String) для элементов, параметров и типов событий.
4. Партиционирование по `session_id`.
5. Удаление данных сессии командой `DROP PARTITION`.
6. HTTP-интерфейс, позволяющий работать без отдельного драйвера.

Последний пункт упростил прототип, но имеет цену: SQL-запросы формируются вручную, а ответы SELECT разбираются из TSV. В промышленной версии безопаснее использовать официальный драйвер ClickHouse с параметризованными запросами.

Партиционирование по каждой сессии удобно для дипломного сценария, где удаление выполняется целиком. Однако при очень большом числе коротких сессий количество партиций может стать избыточным. Для промышленной установки пришлось бы оценить альтернативу: партиционирование по месяцу или дню с дополнительным условием удаления по `session_id`. В текущих тестах число сессий невелико, поэтому простота удаления важнее.

2.4. Выбор формата архива

Архив должен читаться последовательно и не требовать загрузки всего набора данных при записи. Поэтому каждая запись сериализуется как отдельный JSON-объект в формате JSON Lines на основе формата JSON [5]. Файл сжимается `gzip` [6].

Преимущества решения:

1. Архив можно создавать потоково.
2. Повреждение одной строки не меняет структуру всего файла.
3. Формат не зависит от ClickHouse.
4. Файл можно распаковать стандартными средствами.
5. Повторяющиеся имена полей хорошо сжимаются.

Недостаток — отсутствие индекса. В прототипе для выдачи данных архив читается полностью.

Другим вариантом мог быть Parquet. Он лучше подходит для выборочного чтения столбцов и обычно эффективнее по размеру, но требует дополнительной библиотеки и более сложного кода. JSON Lines выбран как понятный промежуточный вариант: архив можно проверить вручную командой `gzip -dc`, а его структура совпадает с моделями API.

2.5. Выбор языка и библиотек

Сервер реализован на Go 1.22 [7]. Стандартная библиотека предоставляет HTTP-сервер, контексты, таймеры, синхронизацию и работу с `gzip`. Для маршрутизации используется `chi` [8], для WebSocket — `gorilla/websocket` [9]. Контейнерный запуск основан на Docker [10].

В проекте всего две внешние Go-зависимости. Это упрощает сборку и уменьшает размер контейнера. Многостадийный Dockerfile компилирует статический бинарный файл, после чего копирует его в образ Alpine.

2.6. Выводы по второй главе

Выбранная схема состоит из WebSocket для входящего потока, REST API для управления, ClickHouse для активных данных и gzip-файлов для архива. Решение компактно и соответствует масштабу дипломного проекта. Эксплуатационные особенности буферизации, формирования SQL и чтения архива рассматриваются в пятой главе.

3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1. Архитектура

Система разделена на четыре слоя:

1. handler принимает HTTP- и WebSocket-запросы.
2. service управляет сессиями, буферами, архивированием и удалением.
3. repository формирует запросы к ClickHouse.
4. db выполняет HTTP-запросы к ClickHouse и разбирает ответы.

Общая схема показана на рисунке 3.1.

Архитектура системы централизованного логирования

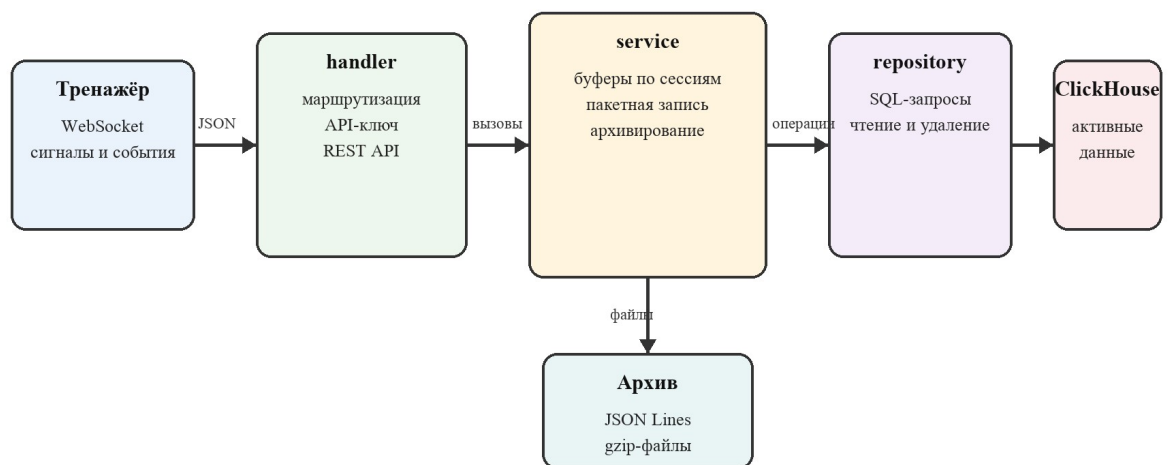


Рисунок 3.1 - Архитектура системы централизованного логирования

Тренажёр не обращается к базе напрямую. Он знает только адрес WebSocket и формат сообщений. Это позволяет изменить способ хранения, не меняя клиентскую часть.

Связь между слоями направлена в одну сторону: handler вызывает service, service — repository, repository — db. Обратных импортов нет. Модели вынесены в отдельный пакет и используются всеми слоями. Такая зависимость упрощает чтение проекта и позволяет в дальнейшем подменить репозиторий в тестах.

3.2. Поток обработки сообщения

Для сигнального сообщения выполняются следующие действия:

1. wsStream читает текстовый WebSocket-кадр.

2. JSON разбирается в структуру WSMessage.
3. По полю type выбирается модель SignalLog.
4. SessionID заменяется значением из URL.
5. AddSignalLog добавляет запись в буфер нужной сессии.
6. При достижении 500 записей вызывается flushSignals.
7. Репозиторий формирует один INSERT с несколькими строками.

Для событий путь аналогичен, но используется отдельный буфер и таблица event_logs.

При закрытии соединения возможны два сценария. В штатном сценарии клиент отправляет session_end, сервер сбрасывает оба буфера и возвращается из обработчика. Если соединение оборвано, цикл чтения завершается нештатно, после чего EndSession вызывается с context.Background(). Такой подход позволяет отделить финальную запись накопленных данных от состояния клиентского соединения и использовать его как основу для устойчивого завершения сессии.

3.3. WebSocket и REST API

WebSocket-маршрут имеет вид:

/sessions/{session_id}/stream?api_key=...

Поддерживаются сообщения signal, event и session_end. Сервер отправляет ping каждые 30 секунд и ожидает pong не более 60 секунд.

Маршруты REST API приведены в таблице 3.1.

Таблица 3.1 - Маршруты API

Метод	Маршрут	Назначение
GET	/health	Проверка запуска сервера
GET	/sessions/{id}/stream	WebSocket-приём сообщений
GET	/sessions/{id}/signal-logs	Получение сигналов
GET	/sessions/{id}/event-logs	Получение событий
POST	/sessions/{id}/archive	Создание архива

Метод	Маршрут	Назначение
DELETE	/sessions/{id}	Полное удаление сессии

Для чтения поддерживаются параметры `limit`, `offset` и `restart_id`. Репозиторий ограничивает `limit` значением 10 000. Если параметр отсутствует или некорректен, используется 1000.

Ответ для сигналов и событий имеет одинаковую форму: идентификатор сессии, массив `logs` и общее число `total`. Это удобно для интерфейса с страничной загрузкой. Для активной базы `total` считается отдельным запросом `count()`. Для архива поле равно числу объектов, прочитанных из файла.

3.4. Модель данных

Структура таблиц приведена в таблице 3.2.

Таблица 3.2 - Таблицы ClickHouse

Таблица	Основные поля	Движок и ключ
sessions	id, started_at, ended_at, status	ReplacingMergeTree; ORDER BY id
signal_logs	session_id, restart_id, element, parameter, model_time, value	MergeTree; PARTITION BY session_id
event_logs	id, session_id, restart_id, username, event_type, model_time, data	MergeTree; PARTITION BY session_id

Таблицы `signal_logs` и `event_logs` партиционированы по `session_id`. Сортировка (`session_id`, `restart_id`, `model_time`) совпадает с основным способом чтения данных. Таблица `sessions` использует `ReplacingMergeTree` и хранит статус сессии.

Поле `id` событий генерируется функцией `generateUUIDv4()` на стороне ClickHouse. Для сигналов отдельный идентификатор не создаётся, поскольку запись определяется сочетанием сессии, перезапуска, элемента, параметра и

модельного времени. Уникальное ограничение не задано: сервер исходит из того, что источник не отправляет дубликаты.

3.5. Буферизация

В сервисе используются две карты:

```
map[uint64][]model.SignalLog
```

```
map[uint64][]model.EventLog
```

Ключом является `session_id`. Доступ защищён отдельными `sync.Mutex`, поэтому запись сигналов не блокирует работу с событиями.

Сброс выполняется:

1. При накоплении 500 записей.
2. По таймеру один раз в секунду.
3. При завершении WebSocket-сессии.
4. Перед чтением логов.
5. Перед архивированием.

Перед записью данные копируются во временный срез, а исходный буфер очищается. Блокировка освобождается до обращения к ClickHouse, поэтому сетевой запрос не удерживает mutex.

Одновременно для одной сессии могут сработать пороговый и периодический flush. Mutex не позволяет двум операциям забрать один и тот же срез. Первая операция очищает буфер, вторая видит нулевую длину и завершается. Новые сообщения, поступившие во время SQL-запроса, попадают уже в новый буфер и будут записаны следующей операцией.

3.6. Архивирование

Архивирование запускается запросом `POST /sessions/{id}/archive`. Сначала сервис сбрасывает оба буфера. Затем данные читаются порциями по 5000 записей и записываются в два файла:

```
signal_logs_{session_id}.jsonl.gz
```

```
event_logs_{session_id}.jsonl.gz
```

После создания файлов статус в оперативной памяти меняется на `archived`. Обновление статуса в ClickHouse и удаление партиций выполняются в фоновой горутине. Это уменьшает время ответа API, но создаёт промежуток, в котором архив уже доступен, а активные записи ещё удаляются.

Статус дополнительно хранится в карте `statuses`. Это сделано из-за особенностей мутаций ClickHouse: команда `ALTER TABLE ... UPDATE` может применяться не мгновенно. Сразу после архивирования сервис уже должен читать файл, поэтому он не ждёт обновления таблицы. После перезапуска процесса карта пуста, и статус снова определяется запросом `GetSession`.

3.7. Удаление

Удаление состоит из четырёх шагов:

1. Удаление партиций сигналов и событий.
2. Удаление строки из `sessions`.
3. Удаление двух архивных файлов.
4. Очистка карт буферов и статусов в памяти.

Единицей удаления остаётся `session_id`. Данные других тренировок не затрагиваются.

3.8. Конфигурация и запуск

Параметры сервера задаются переменными окружения: `SERVER_PORT`, `API_KEY`, `CH_ADDR`, `CH_DB`, `CH_USER`, `CH_PASSWORD`, `ARCHIVE_DIR`.

Docker Compose запускает два сервиса. ClickHouse публикует HTTP-порт 8123 и нативный порт 9000. logflow публикует порт 8080 и подключает отдельный том `/archives`. При старте сервер делает до десяти попыток подключения к ClickHouse с паузой три секунды, затем применяет миграцию `001_init.sql`.

Данные ClickHouse и архивы находятся в разных Docker volumes. Пересоздание контейнера не удаляет их, пока не используется `docker compose down -v`. Миграция содержит `CREATE TABLE IF NOT EXISTS`, поэтому повторный запуск не создаёт дубликаты таблиц.

3.9. Безопасность

Все маршруты, кроме /health, проверяют API-ключ. Для REST он передаётся в X-API-Key или Authorization: Bearer. Для WebSocket допускается query-параметр.

Это базовая защита. В прототипе нет TLS, ролей пользователей и аудита административных действий. Кроме того, CheckOrigin всегда возвращает true. Для закрытой демонстрационной среды это допустимо, но для внешнего развёртывания требует изменения.

3.10. Выводы по третьей главе

Архитектура повторяет фактическую структуру проекта и не содержит лишних промежуточных компонентов. Основные решения — отдельные буферы, пакетный INSERT, партиции по сессии и два архивных файла — направлены на конкретные операции дипломной системы.

4. РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ

4.1. Структура проекта

Практическая часть расположена в каталоге `logflow3`. Назначение основных файлов приведено в таблице 4.1.

Таблица 4.1 - Назначение файлов проекта

Файл	Назначение
<code>cmd/server/main.go</code>	Запуск сервера и graceful shutdown
<code>internal/handler/handler.go</code>	Маршруты, API-ключ, WebSocket
<code>internal/service/service.go</code>	Буферы, архивирование, удаление
<code>internal/repository/repository.go</code>	SQL-запросы к ClickHouse
<code>internal/db/db.go</code>	HTTP-клиент ClickHouse и миграции
<code>internal/model/model.go</code>	Структуры сообщений и ответов
<code>cmd/loadtest/main.go</code>	Генератор нагрузки
<code>migrations/001_init.sql</code>	Создание трёх таблиц

Точка входа содержит только сборку зависимостей и запуск сервера. Работа с HTTP не смешивается с SQL, а архивирование не находится в обработчиках маршрутов.

4.2. Запуск сервера

Функция `main` последовательно выполняет:

1. `config.Load()` — чтение переменных окружения.
2. `db.Connect()` — подключение к ClickHouse.
3. `db.RunMigrations()` — создание таблиц.
4. `repository.New()` — создание слоя данных.
5. `service.New()` — запуск сервисного слоя и фонового flush-цикла.
6. `handler.New()` — настройку маршрутов и API-ключа.
7. `ListenAndServe()` — запуск HTTP-сервера.

Для корректного завершения обрабатываются `SIGINT` и `SIGTERM`. Сервер получает 30 секунд на завершение активных запросов.

HTTP-сервер задаёт ReadTimeout и WriteTimeout по 30 секунд. Middleware с ограничением 60 секунд применяется к health-check и REST-маршрутам, но исключён из WebSocket-маршрута. После upgrade долгоживущее соединение управляется библиотекой gorilla/websocket и механизмом ping/pong. Разделение было выполнено после высокого профиля, в котором общий HTTP timeout отменял финальный flush на 60-й секунде.

Обработчик сигнала завершения находится в отдельной горутине. После получения сигнала создаётся контекст с таймаутом 30 секунд и вызывается srv.Shutdown. Новые соединения перестают приниматься, а действующие обработчики получают время на завершение. При этом фоновый flushLoop не имеет механизма остановки, поскольку сервис не реализует метод Close. Для прототипа процесс завершается вместе с приложением, но в более строгой реализации цикл нужно завершать через канал или контекст.

4.3. Клиент ClickHouse

CHClient использует HTTP-интерфейс ClickHouse. Метод do создаёт POST-запрос, передаёт имя базы и пользователя в query-параметрах и помещает SQL в тело запроса.

Для команд без результата используется Exec. Для выборок метод Query добавляет FORMAT TSV, разбивает ответ по строкам и табуляциям. Это решение занимает мало кода, но переносит преобразование типов в репозиторий.

При старте Connect проверяет маршрут /ping. Контейнер ClickHouse может запускаться дольше приложения, поэтому сервер выполняет несколько попыток подключения с паузой между ними. После успешного ответа применяются миграции. Такой порядок делает запуск контейнерного окружения устойчивым к разнице во времени старта сервисов.

Миграционный механизм читает файл целиком, пропускает строки-комментарии и считает выражение завершённым, если строка заканчивается точкой с запятой. Для текущей миграции этого достаточно. Такой разбор не

поддерживает сложный SQL с точками с запятой внутри строк или процедур, но в схеме используются только три простых CREATE TABLE.

Метод Ping выполняет обычный GET /ping без передачи имени пользователя и пароля. При конфигурации ClickHouse с обязательной аутентификацией это может отличаться от условий основных запросов. Более точная проверка должна использовать тот же метод do, что и рабочие SQL-команды.

4.4. Обработка WebSocket

Маршрут создаётся в Handler.Router. После проверки ключа функция wsStream извлекает session_id через chi.URLParam и переводит HTTP-соединение в WebSocket.

После подключения вызывается StartSession. Репозиторий вставляет строку в таблицу sessions со статусом active. Далее запускается pingLoop, а основной цикл читает сообщения.

Перед upgrade запрос уже проходит authMiddleware. Сначала ключ ищется в api_key, затем в заголовке X-API-Key, затем в Authorization: Bearer. Сравнение выполняется обычной операцией key != h.apiKey. Для единственного статического ключа этого достаточно, однако конфигурация по умолчанию dev-secret-key не должна использоваться во внешнем окружении.

Разбор выполняется в два этапа. Сначала читается оболочка:

```
type WSMessage struct {  
    Type string    json:"type"  
    Data json.RawMessage json:"data"  
}
```

Затем поле Data преобразуется в SignalLog или EventLog. Такой вариант проще, чем единая структура с большим количеством необязательных полей.

Некорректное сообщение фиксируется в серверном журнале и пропускается. Соединение остаётся активным, поэтому единичная некорректная запись не прерывает передачу остальных данных. Неизвестный type также не завершает сессию.

Размер входящего сообщения явно не ограничен. `ReadBufferSize` влияет на внутренний буфер библиотеки, но не задаёт максимальный размер кадра. Для защиты от случайно большого JSON следует вызвать `SetReadLimit`. В текущих тестах сообщения имеют размер в пределах нескольких сотен байт.

Ping отправляется из отдельной горутины. Нагрузочный клиент постоянно читает соединение, поэтому библиотека обрабатывает служебные кадры и отправляет pong в длительных сессиях. Если сервер начнёт передавать прикладные сообщения клиенту, конкурентную запись в `websocket.Conn` потребуется синхронизировать.

4.5. Реализация буферов

Размер буфера задан константой:

```
const (  
    flushSize    = 500  
    flushInterval = time.Second  
)
```

`AddSignalLog` блокирует `signalMu`, добавляет запись и сразу освобождает `mutex`. Если размер достиг 500, вызывается `flushSignals`.

В `flushSignals` данные сначала копируются:

```
cp := make([]model.SignalLog, len(s.signalBuf[sessionID]))  
copy(cp, s.signalBuf[sessionID])  
s.signalBuf[sessionID] = s.signalBuf[sessionID][:0]
```

После разблокировки временный срез передаётся в репозиторий. Такой порядок уменьшает время блокировки и позволяет продолжать приём новых сообщений во время обращения к `ClickHouse`. Для повышения устойчивости финальной записи целесообразно выполнять сброс буфера в отдельном контексте и сохранять возможность повторной вставки при нештатном ответе хранилища.

Контекст передаётся от `WebSocket`-обработчика в сервис без изменения. Пока соединение активно, это позволяет отменить зависший SQL-запрос вместе с запросом клиента. При завершении сессии такое поведение спорно: финальная

запись важнее состояния соединения. Для неё лучше создавать отдельный контекст с небольшим таймаутом, например 10 секунд.

Фоновый `flushLoop` раз в секунду получает список идентификаторов активных сессий и вызывает сброс обоих типов данных. Для полноты периодического сброса список должен формироваться по объединению буферов сигналов и событий. Тогда даже сессия, содержащая только события, будет записываться по таймеру.

После успешного `flush` в карте остаётся пустой срез. Идентификатор не удаляется, поэтому список известных сессий со временем растёт. При небольшом количестве тренировок это незаметно, но длительно работающий сервер должен удалять пустые элементы после завершения или удаления сессии.

4.6. Пакетная запись

`InsertSignalLogs` формирует один SQL-запрос:

```
INSERT INTO signal_logs  
    (session_id, restart_id, element, parameter, model_time, value)  
VALUES (...), (...), ...
```

При размере буфера 500 один HTTP-запрос заменяет 500 отдельных вставок. Для профиля с 297 750 сигналами это уменьшает порядок числа запросов с сотен тысяч до сотен.

Числовые значения форматируются через `%f`, то есть с шестью знаками после запятой. Для текущей генерации этого достаточно. Если потребуется сохранять больше значащих цифр `float64`, можно перейти на форматирование с расширенной точностью или использовать нативный драйвер.

Строки экранируются функцией `chStr`, которая заменяет одинарную кавычку. Это достаточно для текущих тестовых данных, но не является полной заменой параметризованных запросов.

В событиях поле `data` хранится как строка. Репозиторий не проверяет, содержит ли она корректный JSON. Это решение допускает разные структуры

событий, но перекладывает проверку на источник. Альтернативой является тип JSON или проверка `json.Valid` в обработчике.

4.7. Чтение данных

Получение сигналов выполняется двумя запросами. Первый вычисляет `count()`, второй возвращает страницу:

```
SELECT session_id, restart_id, element, parameter, model_time, value
FROM signal_logs
WHERE session_id = ...
ORDER BY restart_id, model_time
LIMIT ... OFFSET ...
```

Если передан `restart_id`, он добавляется в `WHERE`. Такой же подход используется для событий.

Перед чтением сервис сбрасывает соответствующий буфер. Поэтому REST-клиент видит сообщения, которые были приняты недавно и ещё не попали в периодический `flush`.

Параметр `offset` не имеет верхнего ограничения. Большое смещение заставляет ClickHouse пройти предыдущие строки, поэтому для длинных сессий лучше использовать курсор по `(restart_id, model_time)` вместо классической пагинации. Для дипломного API `limit/offset` проще и понятнее, но это компромисс.

Если сессия не найдена, репозиторий возвращает пустой массив и `total=0`, а HTTP-код остаётся 200. Для единообразия API архивный путь также можно привести к политике 404 для неизвестной сессии. Это сделает поведение маршрутов более последовательным для клиента.

4.8. Создание и чтение архива

Функции `archiveSignals` и `archiveEvents` читают ClickHouse порциями по 5000 строк. Для каждой записи `json.Encoder` создаёт отдельную строку, а `gzip.Writer` сразу сжимает поток.

При чтении используется `gzip.NewReader` и `json.Decoder`. Внешний формат ответа не меняется: клиент получает `session_id`, `logs` и `total`.

В проверочном запуске время ответа составило 0,124 секунды. Для более крупного архива объём ответа и расход памяти будут расти линейно.

Функции архивации используют defer для закрытия файла и gzip-writer. Для повышения атомарности архив можно сначала создавать во временном файле с расширением .tmp, закрывать его, а затем переименовывать в итоговое имя. Тогда наличие файла будет означать полностью сформированный архив.

Архивирование сигналов и событий выполняется последовательно. HTTP-клиент ожидает сброса буферов, чтения записей из ClickHouse и полного создания обоих gzip-файлов. Только после этого обработчик возвращает HTTP 200 со статусом archived.

После создания файлов сервис немедленно меняет статус в памяти, а затем запускает фоновую goroutine для обновления статуса в ClickHouse и удаления партиций signal_logs и event_logs. Клиент эту стадию не ожидает и отдельного уведомления о её завершении не получает; ошибки записываются только в серверный журнал.

REST-маршрут ограничен timeout 60 секунд. Поэтому при дальнейшем росте объёма синхронное формирование архива может быть отменено до ответа. Для крупных сессий целесообразно запускать архивирование как фоновое задание и возвращать клиенту идентификатор операции с состояниями queued, running, completed и failed.

4.9. Удаление сессии

DeleteSessionLogs выполняет DROP PARTITION отдельно для сигналов и событий. Затем DeleteSession удаляет метаданные. Сервис удаляет файлы через os.Remove и очищает три карты в памяти.

В функциональном тесте удаление архивной сессии заняло 0,015 секунды. После запроса оба файла отсутствовали, а SELECT count() FROM sessions FINAL WHERE id=1009 вернул 0.

Результат os.Remove можно обрабатывать точнее: отсутствие файла считать допустимым для идемпотентности, а проблемы прав доступа возвращать клиенту.

Такой подход сохраняет удобство повторного удаления и одновременно улучшает диагностику.

4.10. Генератор нагрузки

Утилита `cmd/loadtest` создаёт отдельную горутину и WebSocket-соединение для каждой сессии. Сигналы отправляются пакетами по 50 сообщений. Частота задаётся флагами `signals-per-sec`, `events-per-sec`, `sessions` и `duration`. В `Makefile` добавлен профиль `loadtest-hour`: 10 сессий, по 500 сигналов и 20 событий в секунду на сессию в течение одного часа.

Счётчики реализованы через `atomic.Int64`. Каждые пять секунд программа выводит число отправленных сообщений и среднюю скорость. После завершения каждой сессии отправляется `session_end`.

Генератор фиксирует сетевые сбои отправки и отдельно выводит клиентские счётчики. Фоновый цикл чтения WebSocket необходим для обработки `ping/pong` в длительных соединениях. Для полноценной проверки счётчики дополнительно сравниваются с числом строк в ClickHouse.

Начало всех сессий немного сдвигается: между запуском горутин есть пауза 100 миллисекунд. Из-за этого последние сессии работают чуть меньше общего значения `duration`, которое отсчитывается от единого `startTime`. Поэтому фактическое число сообщений ниже простого произведения частоты, количества сессий и времени. Это не ошибка сервера и учитывается сравнением с фактическими клиентскими счётчиками.

Сигнальный таймер отправляет по 50 сообщений за одно срабатывание. При 500 сигналах в секунду интервал равен 100 миллисекундам. Такой режим создаёт короткие всплески, близкие к поведению клиента, который сам собирает данные небольшими пачками.

4.11. Выводы по четвёртой главе

Реализация соответствует спроектированным слоям и успешно собирается командой `go test ./...` Раздельная политика `timeout` и обработка `ping/pong` проверены часовым нагрузочным профилем. Оставшиеся направления развития связаны с

возвратом буфера при произвольной ошибке репозитория, объединением списков активных буферов и постраничным чтением архива.

5. ТЕСТИРОВАНИЕ СИСТЕМЫ

5.1. Тестовая среда

Тестирование выполнялось в локальном контейнерном окружении Docker. Для проверки использовались:

1. Go 1.22.
2. ClickHouse 24.3-alpine.
3. Образ Alpine 3.19 для сервера.
4. Docker Compose из файла проекта.
5. API на localhost:8080.

Перед каждым нагрузочным профилем таблицы `signal_logs`, `event_logs` и `sessions` очищались. Благодаря этому результаты профилей не смешивались.

Контейнеры собирались из текущего состояния каталога `logflow3`. После старта проверялись состояние сервисов и серверный журнал. ClickHouse переходил в состояние `healthy`, а приложение сообщало об успешном применении миграций. Генератор нагрузки запускался с хостовой системы и подключался к опубликованному порту 8080.

Размер данных в ClickHouse определялся по `system.parts`: суммировались `data_uncompressed_bytes` активных частей таблиц сигналов и событий. Размер архивов измерялся внутри контейнера `logflow` командой `wc -c /archives/*.gz`. Такой способ позволяет повторить измерения без дополнительного программного обеспечения.

5.2. Функциональная проверка

Результаты функциональных запросов приведены в таблице 5.1.

Таблица 5.1 - Результаты функциональной проверки

Проверка	Результат	Время / объём
GET /health	HTTP 200	0,0037 с; 16 байт
Запрос без API-ключа	HTTP 401	0,0029 с; 39 байт
Чтение архивной сессии с <code>limit=10</code>	HTTP 200; получен полный архив	0,124 с; 3,32 МБ

Проверка	Результат	Время / объём
DELETE архивной сессии	HTTP 200; файлы и метаданные удалены	0,015 с
go test ./...	Сборка всех пакетов успешна	Сборка подтверждена

Проверка показала, что API-ключ обрабатывается корректно: запрос без ключа получил код 401 за 0,0029 секунды. Удаление очистило метаданные и архивы. При чтении архивной сессии зафиксирована особенность текущей реализации: архив возвращается целиком, а параметры постраничной выдачи применяются только к активным данным.

Кроме HTTP-ответа проверялось состояние хранилища. После удаления сессии 1009 оба gzip-файла отсутствовали, а запрос к sessions FINAL вернул ноль строк. Это подтверждает, что операция затрагивает не только внешний API, но и все места хранения.

Команда go test ./... завершилась успешно для восьми пакетов проекта. Результат подтверждает сборку, но не проверяет поведение: каждый пакет вывел [no test files]. Поэтому основная проверка выполнялась интеграционно через запущенные контейнеры.

5.3. Методика нагрузочного теста

Использовались четыре профиля из Makefile. Лёгкий профиль проверяет базовый сценарий. Средний увеличивает суммарный поток до 1500 сигналов в секунду. Высокий запускает 10 соединений и целевой поток 5000 сигналов в секунду в течение 60 секунд. Часовой профиль повторяет высокую нагрузку 3600 секунд и используется как регрессионная проверка исправления.

Для каждого профиля фиксировались:

1. Фактическое время работы.
2. Число отправленных сигналов и событий.
3. Средняя скорость передачи.
4. Клиентские ошибки.

5. Число строк в ClickHouse.
6. Размер данных в активной базе.
7. Размер gzip-архивов.

Количество сохранённых строк проверялось отдельными SQL-запросами. Это важно, поскольку клиентская отправка и фактическая запись в ClickHouse относятся к разным этапам обработки.

После каждого теста выполнялся запрос количества строк по каждой сессии. Высокий профиль локализовал потерю последней порции сессии 1000 из-за общего HTTP timeout. После изменения маршрутизации и обработки ping/pong часовой профиль проверил исправление на длительности, в 60 раз большей.

5.4. Результаты нагрузки

Результаты четырёх профилей приведены в таблице 5.2.

Таблица 5.2 - Результаты нагрузочных тестов

Профиль	Сессии	Отправлено сигналов	Скорость, сигн./с	Сохранено	Расхождение
Лёгкий	3	5 950	296	5 950	0
Средний	5	44 600	1 486	44 600	0
Высокий	10	297 750	4 961	297 450	300 (0,101%)
1 час	10	≈18 млн	4 999	≈18 млн	0

Фактическая скорость показана на рисунке 5.1.

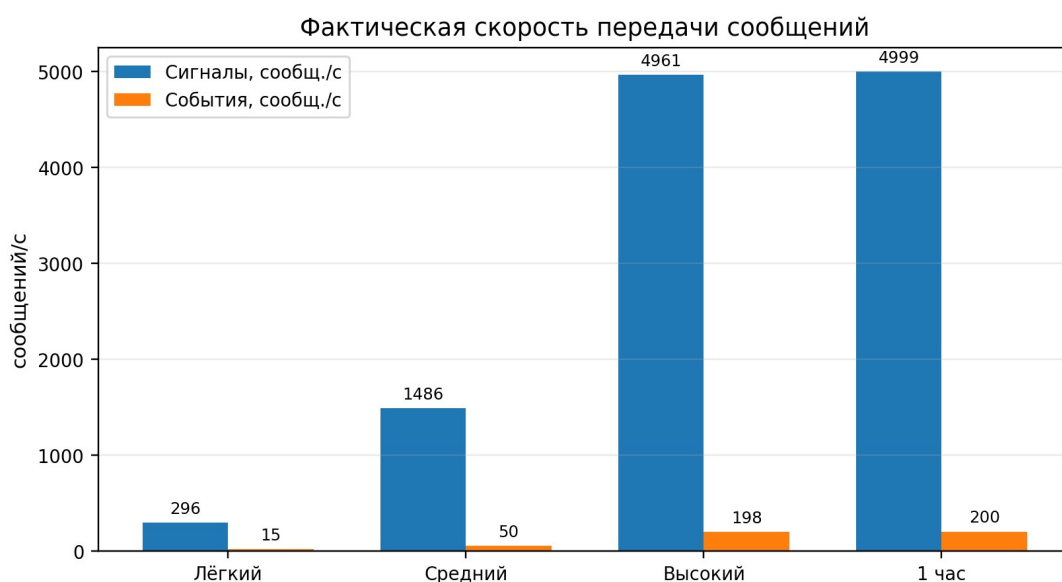


Рисунок 5.1 - Фактическая скорость передачи сообщений

Лёгкий профиль отправил 5950 сигналов и 297 событий. Все записи были найдены в ClickHouse. Средний профиль также завершился без расхождений: 44 600 сигналов и 1489 событий.

В лёгком профиле средняя скорость составила 296 сигналов и 15 событий в секунду. Она близка к заданным 300 и 15 сообщениям в секунду. Небольшое отличие связано со сдвигом запуска горутин и завершением по общему времени.

Средний профиль дал 1486 сигналов и 50 событий в секунду при целевых значениях 1500 и 50. Отклонение по сигналам составило менее одного процента.

Серверный журнал не содержал сбоев flush, а число строк полностью совпало с клиентскими счётчиками.

Высокий профиль передал 297 750 сигналов со средней скоростью 4961 сигнал в секунду. Клиент не зафиксировал сетевых сбоев. В ClickHouse оказалось 297 450 сигналов и 11 890 событий. Расхождение составило 300 сигналов и 10 событий.

До завершения исходный высокий профиль выполнялся стабильно. Расхождение сформировалось только при финальном сбросе одной сессии и относится не к пропускной способности ClickHouse, а к политике завершения WebSocket: общий `middleware.Timeout(60 * time.Second)` отменял контекст WebSocket-запроса на 60-й секунде, поэтому финальный flush последней порции не успевал завершиться. После исключения WebSocket-маршрута из общего timeout и корректной обработки ping/pong ошибка не повторилась в часовом профиле.

После точечного исправления выполнен четвёртый профиль. За 3600 секунд отправлено 17 995 850 сигналов и 719 665 событий. Средняя скорость составила 4999 сигналов и около 200 событий в секунду. Все клиентские счётчики совпали с числом строк в ClickHouse, ошибок и сообщений `context deadline exceeded` не было.

Пиковое наблюдаемое потребление памяти ClickHouse составило около 2,67 ГиБ при лимите Docker Desktop 7,75 ГиБ; Go-сервер использовал около 20 МиБ. Следовательно, часовой профиль поместился в выделенную память с запасом примерно 5 ГиБ, хотя объём хранилища продолжает расти линейно.

Полнота сохранения показана на рисунке 5.2.

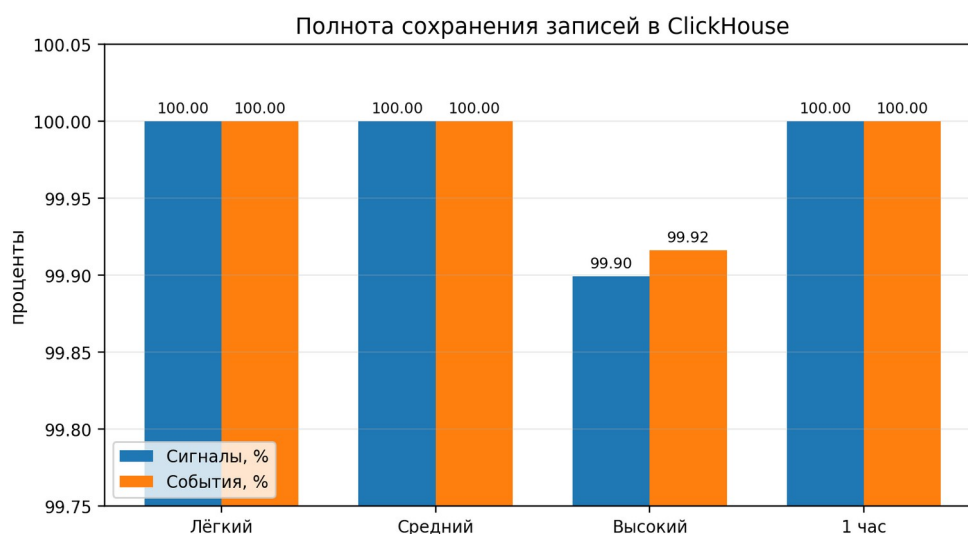


Рисунок 5.2 - Полнота сохранения сообщений

В исходном высоком профиле расхождение составило 0,101% сигналов и 0,084% событий. В часовом профиле после исправления полнота составила 100% для обоих типов сообщений. Это подтверждает, что третий тест корректно выявил ошибку завершения, а четвёртый проверил её устранение.

Показатели клиента и базы для высокого профиля различаются, при этом клиентский счётчик сетевых сбоев равен нулю. WebSocket-запись была успешной, поэтому сообщения дошли до обработчика или сетевого буфера. Расхождение относится к границе сервисного слоя и ClickHouse, что подтверждает пользу отдельной проверки базы.

5.5. Архивирование и сжатие

Размеры активных данных и архивов приведены в таблице 5.3.

Таблица 5.3 - Результаты архивирования

Профиль	Несжатые данные, байт	Архивы gzip, байт	Уменьшение
Лёгкий	269 671	84 524	68,7%
Средний	1 846 925	586 483	68,2%
Высокий	12 748 100	3 965 401	68,9%
Часовой, 1 сессия	81 247 397	25 251 962	68,9%

Сравнение объёма показано на рисунке 5.3.

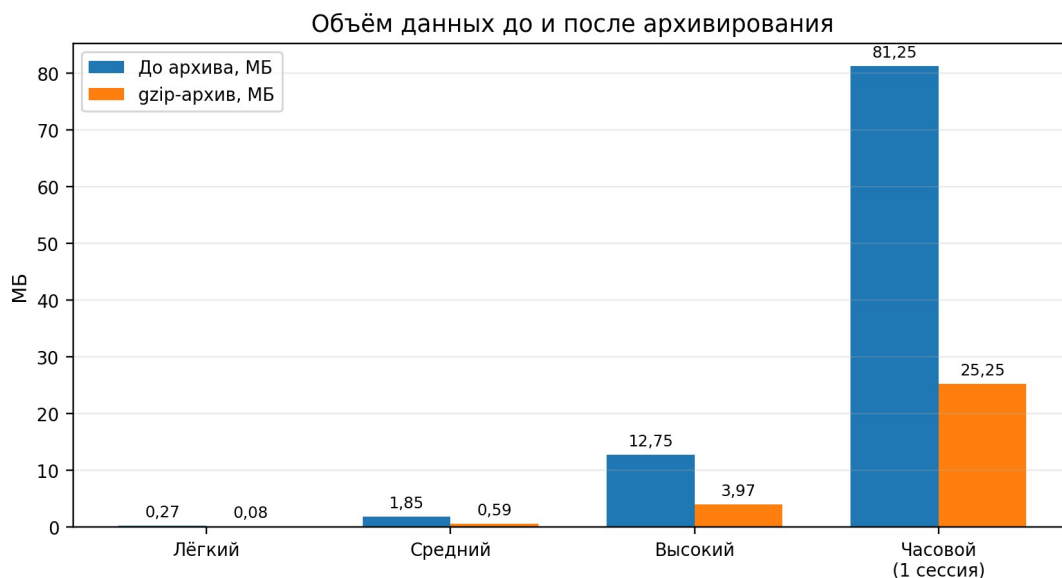


Рисунок 5.3 - Объём данных до и после архивирования

Во всех профилях gzip уменьшил объём примерно на 68–69% относительно несжатого размера данных ClickHouse. Для одной сессии часового профиля 81,25 МБ несжатых данных превратились в два архива общим размером 25,25 МБ.

Сравнение не учитывает служебные файлы и метаданные ClickHouse, поэтому его следует рассматривать как оценку данных, а не полный размер каталога базы. Тем не менее результат подтверждает целесообразность архивного слоя.

Коэффициент уменьшения вычислялся по формуле:

$$R = (1 - S_{\text{archive}} / S_{\text{source}}) \cdot 100 \%, \quad (5.1)$$

где S_{source} — суммарный несжатый размер активных частей двух таблиц, а S_{archive} — размер двух gzip-файлов всех сессий профиля.

Близкий процент во всех четырёх измерениях объясняется одинаковой структурой тестовых записей. Для часового профиля в таблицу и диаграмму включена одна сессия, а не сумма десяти сессий. Для реальных данных коэффициент может отличаться, особенно при больших уникальных строках.

Для оценки ожидания клиента архивировались две большие сессии. Сессия с 1 896 850 сигналами и 75 877 событиями была обработана за 19,94 секунды. Сессия с 1 897 250 сигналами и 75 871 событием — за 29,12 секунды. В обоих случаях HTTP 200 был получен только после формирования двух файлов.

После ответа обновление статуса и DROP PARTITION выполнялись асинхронно. По журналу ClickHouse три фоновые команды заняли 8, 10 и 7 мс. Отдельного уведомления о завершении фоновой стадии нет.

5.6. Выводы по пятой главе

Система успешно обработала лёгкий и средний профили без потерь. Исходный высокий профиль достиг 4961 сигнала в секунду и выявил конфликт WebSocket с общим HTTP timeout. После точечного исправления часовой профиль сохранил 17 995 850 сигналов и 719 665 событий без расхождений при средней скорости 4999 сигналов в секунду.

Архивирование уменьшает объём данных примерно на две трети. Формирование файлов выполняется синхронно и заняло 19,94–29,12 секунды, после чего статус и удаление партиций обрабатываются в фоне без отдельного уведомления. Для более крупных сессий требуется модель фоновых заданий.

ПРИЛОЖЕНИЕ 1. ПРИМЕРЫ WEBSOCKET-СООБЩЕНИЙ

Сигнал:

```
{
  "type": "signal",
  "data": {
    "session_id": 1000,
    "restart_id": 1,
    "element": "H1S3S321",
    "parameter": "U",
    "model_time": 10.2,
    "value": 231.7
  }
}
```

Событие:

```
{
  "type": "event",
  "data": {
    "session_id": 1000,
    "restart_id": 1,
    "username": "ivanovii",
    "event_type": "Alarm",
    "model_time": 11.4,
    "data": "{ \"area\": \"Котел\", \"message\": \"Тестовое событие\" }"
  }
}
```

Завершение сессии:

```
{
  "type": "session_end",
  "data": {
```

```
"session_id": 1000
}
}
```

ПРИЛОЖЕНИЕ 2. КОМАНДЫ ФУНКЦИОНАЛЬНОЙ ПРОВЕРКИ

```
docker compose up -d --build
```

```
curl http://localhost:8080/health
```

```
make loadtest-light
```

```
curl -H "X-API-Key: dev-secret-key" \  
  "http://localhost:8080/sessions/1000/signal-logs?limit=10"
```

```
curl -X POST -H "X-API-Key: dev-secret-key" \  
  "http://localhost:8080/sessions/1000/archive"
```

```
curl -X DELETE -H "X-API-Key: dev-secret-key" \  
  "http://localhost:8080/sessions/1000"
```

ПРИЛОЖЕНИЕ 3. ФРАГМЕНТЫ СХЕМЫ CLICKHOUSE

```
CREATE TABLE IF NOT EXISTS signal_logs (  
    session_id UInt64,  
    restart_id UInt64,  
    element LowCardinality(String),  
    parameter LowCardinality(String),  
    model_time Float64,  
    value Float64  
)  
ENGINE = MergeTree()  
PARTITION BY session_id  
ORDER BY (session_id, restart_id, model_time);  
  
CREATE TABLE IF NOT EXISTS event_logs (  
    id UUID DEFAULT generateUUIDv4(),  
    session_id UInt64,  
    restart_id UInt64,  
    username LowCardinality(String),  
    event_type LowCardinality(String),  
    model_time Float64,  
    data String  
)  
ENGINE = MergeTree()  
PARTITION BY session_id  
ORDER BY (session_id, restart_id, model_time);
```

ПРИЛОЖЕНИЕ 4. ЖУРНАЛ НАГРУЗОЧНЫХ ТЕСТОВ

Высокий профиль до исправления; сессий: 10

Сигналов/сек на сессию: 500

Событий/сек на сессию: 20

Длительность: 60 секунд

Сигналов отправлено: 297750

Событий отправлено: 11900

Клиентских ошибок: 0

Сигналов сохранено: 297450

Событий сохранено: 11890

Часовой профиль после исправления

Сессий: 10; длительность: 3600 секунд

Сигналов отправлено и сохранено: 17 995 850

Событий отправлено и сохранено: 719 665

Средняя скорость: 4999 сигналов/с и 200 событий/с

Клиентских ошибок: 0

Пиковая память: ClickHouse 2,67 ГиБ; сервер около 20 МиБ

Архивация одной сессии: 1 897 250 сигналов и 75 871 событие; 29,12 с; gzip
25 251 962 байта

ЗАКЛЮЧЕНИЕ

В выпускной работе разработана система централизованного логирования тренажёрных сессий. Сервер принимает сигналы и события через WebSocket, накапливает их в отдельных буферах и записывает пакетами в ClickHouse. Для чтения, архивирования и удаления реализован REST API.

Практическая часть включает Go-сервис, миграцию базы данных, Dockerfile, Docker Compose и генератор нагрузки. Схема таблиц учитывает идентификатор сессии, перезапуск и модельное время. Партиционирование по session_id позволяет удалять данные тренировки без построчной обработки.

Функциональные тесты подтвердили запуск окружения, проверку API-ключа, получение данных, архивирование и полное удаление сессии. Исходный высокий профиль выявил потерю около 0,1% данных из-за отмены контекста на 60-й секунде. После исключения WebSocket из обычного HTTP timeout и обработки ping/pong часовой профиль сохранил 17 995 850 сигналов и 719 665 событий без потерь.

Полученные результаты показывают, что система устойчиво обрабатывает около 5 тыс. сигналов в секунду в течение одного часа. ClickHouse использовал до 2,67 ГиБ памяти из 7,75 ГиБ, а сервер — около 20 МиБ.

Архивирование в JSON Lines с gzip уменьшило объём большой сессии на 68,9%. Клиент синхронно ожидает формирования файлов 19,94–29,12 секунды. Обновление статуса и удаление активных партиций выполняются после ответа асинхронно, отдельного уведомления об их завершении нет.

Цель работы достигнута на уровне программной реализации. Реализован и экспериментально проверен полный цикл данных: часовой приём, пакетная запись, чтение, архивирование большого набора и удаление. Четвёртый нагрузочный профиль подтвердил исправление обнаруженного timeout и позволил определить следующий приоритет — фоновую архивацию с наблюдаемым состоянием.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Kleppmann, M. Designing Data-Intensive Applications / M. Kleppmann. — Sebastopol: O'Reilly Media, 2017. — 616 p.
2. Newman, S. Building Microservices: Designing Fine-Grained Systems / S. Newman. — 2nd ed. — Sebastopol: O'Reilly Media, 2021. — 616 p.
3. The WebSocket Protocol: RFC 6455. — Текст: электронный // RFC Editor: [сайт]. — URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обращения: 20.05.2026).
4. ClickHouse Documentation. — Текст: электронный // ClickHouse: [сайт]. — URL: <https://clickhouse.com/docs/> (дата обращения: 20.05.2026).
5. The JavaScript Object Notation Data Interchange Format: RFC 8259. — Текст: электронный // RFC Editor: [сайт]. — URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обращения: 20.05.2026).
6. GZIP file format specification: RFC 1952. — Текст: электронный // RFC Editor: [сайт]. — URL: <https://www.rfc-editor.org/rfc/rfc1952> (дата обращения: 20.05.2026).
7. The Go Programming Language. Documentation. — Текст: электронный // Go: [сайт]. — URL: <https://go.dev/doc/> (дата обращения: 20.05.2026).
8. Chi router documentation. — Текст: электронный // GitHub: [сайт]. — URL: <https://github.com/go-chi/chi> (дата обращения: 20.05.2026).
9. Gorilla WebSocket package documentation. — Текст: электронный // GitHub: [сайт]. — URL: <https://github.com/gorilla/websocket> (дата обращения: 20.05.2026).
10. Docker Documentation. — Текст: электронный // Docker Docs: [сайт]. — URL: <https://docs.docker.com/> (дата обращения: 20.05.2026).

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	8
1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ.....	11
1.1. Источник данных.....	11
1.2. Типы хранимых данных.....	11
1.3. Функциональные требования.....	12
1.4. Нефункциональные требования.....	12
1.5. Критерии проверки.....	13
1.6. Выводы по первой главе.....	14
2. ВЫБОР ТЕХНИЧЕСКОГО РЕШЕНИЯ.....	15
2.1. Готовая платформа или отдельный сервис.....	15
2.2. Выбор транспорта.....	15
2.3. Выбор хранилища.....	16
2.4. Выбор формата архива.....	17
2.5. Выбор языка и библиотек.....	17
2.6. Выводы по второй главе.....	18
3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ.....	19
3.1. Архитектура.....	19
3.2. Поток обработки сообщения.....	19
3.3. WebSocket и REST API.....	20
3.4. Модель данных.....	21
3.5. Буферизация.....	22
3.6. Архивирование.....	22
3.7. Удаление.....	23
3.8. Конфигурация и запуск.....	23
3.9. Безопасность.....	23
3.10. Выводы по третьей главе.....	24
4. РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ.....	25
4.1. Структура проекта.....	25
4.2. Запуск сервера.....	25
4.3. Клиент ClickHouse.....	26
4.4. Обработка WebSocket.....	27
4.5. Реализация буферов.....	28

4.6. Пакетная запись.....	29
4.7. Чтение данных.....	30
4.8. Создание и чтение архива.....	30
4.9. Удаление сессии.....	31
4.10. Генератор нагрузки.....	32
4.11. Выводы по четвёртой главе.....	32
5. ТЕСТИРОВАНИЕ СИСТЕМЫ.....	34
5.1. Тестовая среда.....	34
5.2. Функциональная проверка.....	34
5.3. Методика нагрузочного теста.....	35
5.4. Результаты нагрузки.....	36
5.5. Архивирование и сжатие.....	39
5.6. Выводы по пятой главе.....	41
ПРИЛОЖЕНИЕ 1. ПРИМЕРЫ WEBSOCKET-СООБЩЕНИЙ.....	42
ПРИЛОЖЕНИЕ 2. КОМАНДЫ ФУНКЦИОНАЛЬНОЙ ПРОВЕРКИ.....	44
ПРИЛОЖЕНИЕ 3. ФРАГМЕНТЫ СХЕМЫ SLICKHOUSE.....	45
ПРИЛОЖЕНИЕ 4. ЖУРНАЛ НАГРУЗОЧНЫХ ТЕСТОВ.....	46
ЗАКЛЮЧЕНИЕ.....	47
БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....	48